

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-  
a5b992c9f86a3dd38.jsonl`  
- SHA-256: `848745b63a680ea7b150c3fac6d3a9337248da96e14bc4e7e8fc51ae707fd5c6`  
- Source modified: `2026-07-06T06:08:40+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:05:01.580Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension tests):

title: No test covers the Tier-2 cross-tier dedup case; the shim silently regresses re-upload dedup and ships uncaught
severity(claimed): high
file:line: tests/test_db.py:402
summary: test_get_video_by_content_fingerprint_hit seeds a video with NO document_id, so no test

exercises the case where a prior item has a document_id and is then looked up by fingerprint ? which the v2 shim gets wrong, re-delivering a duplicate v1 would have suppressed.

failure_scenario: A file is first seen and stored WITH a document_id AND full fingerprint metadata (size/dur/h/w). Its dedup_key becomes 'tg:doc:{id}' (document branch wins in compute_dedup_key). Later the same file is re-encoded/re-uploaded to another channel and arrives with a NEW document.id but identical fingerprint. The ingest path (telethon_client.py:191-221) checks Tier-1 by the new document_id (miss), then Tier-2 by fingerprint via db.get_video_by_content_fingerprint(...). The shim (db.py:1160) computes 'tg:fp:{size}:{dur}:{h}:{w}' and queries content_items by dedup_key ? but the original row's dedup_key is 'tg:doc:...', so the lookup returns None. The re-upload is treated as new and re-delivered. Repro on the actual venv: seed VideoRecord(document_id=555, size=1000,dur=60,h=1080,w=1920), then get_video_by_content_fingerprint(111,1000,60,1080,1920) returns None; v1 (main:db.py:638, raw column query) returns the prior row. No test in test_db.py or test_delivery.py catches this.

evidence: tests/test_db.py:402-425 seeds document_id-absent video only; src/tg_video_filter/db.py:1151-1173 (get_video_by_content_fingerprint shim keys on tg:fp only); src/tg_video_filter/db.py:1215-1224 (insert computes tg:doc when document_id present, tg:fp lost); git show main:src/tg_video_filter/db.py:638-666 (v1 queried raw size/dur/h/w columns, independent of document_id); caller src/tg_video_filter/telethon_client.py:191-221; confirmed by repro returning 'v2 shim fingerprint hit: None'.

recommendation: Add a test that seeds a content item WITH a document_id AND full fingerprint metadata, then asserts get_video_by_content_fingerprint(...) with the SAME fingerprint returns that item (mirroring v1's cross-tier catch). This test fails against the current shim and pins the regression. The fix belongs in db.py (e.g. store both keys, or have the fingerprint shim query the raw content_items metadata columns rather than dedup_key), but the missing test is the review gap here.

2. assistant (2026-07-06T06:05:03.981Z)

I'll verify this finding by reading the actual code at the cited lines and running a repro.

3. user (2026-07-06T06:05:05.493Z)

```

390         inserted = await db.insert_video(
391             conn,
392             VideoRecord(
393                 user_id=USER_ID,
394                 channel_id=CHANNEL_ID,
395                 message_id=21,
396                 document_id=None,
397             ),
398         )
399         assert inserted.id is not None
400
401
402     async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
403         await _seed_user(conn)
404         await db.insert_video(
405             conn,
406             VideoRecord(
407                 user_id=USER_ID,
408                 channel_id=CHANNEL_ID,
409                 message_id=5,
410                 size_bytes=10_000_000,
411                 duration_s=180,
412                 height=720,
413                 width=1280,
414             ),
415         )
416         found = await db.get_video_by_content_fingerprint(
417             conn,
418             USER_ID,
419             10_000_000,
420             180,
421             720,

```

```

422         1280,
423     )
424     assert found is not None
425     assert found.message_id == 5
426
427
428     async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) ->
None:
429         await _seed_user(conn)
430         # No video inserted ? should be None.
431         assert (
432             await db.get_video_by_content_fingerprint(
433                 conn,
434                 USER_ID,
435                 1,
436                 1,
437                 1,
438                 1,
439             )

```

4. user (2026-07-06T06:05:06.138Z)

```

1140     """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141     dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143     if item is None:
1144         return None
1145     src = await get_source_by_id(conn, item.source_id)
1146     if src is None:
1147         return None
1148     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151     async def get_video_by_content_fingerprint(
1152         conn: aiosqlite.Connection,
1153         user_id: int,
1154         size_bytes: int,
1155         duration_s: int,
1156         height: int,
1157         width: int,
1158     ) -> VideoRecord | None:
1159         """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160         dedup_key = compute_dedup_key(
1161             provider="telegram",
1162             size_bytes=size_bytes,
1163             duration_s=duration_s,
1164             height=height,
1165             width=width,
1166         )
1167         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168         if item is None:
1169             return None
1170         src = await get_source_by_id(conn, item.source_id)
1171         if src is None:
1172             return None
1173         return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176     async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
1177         """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178         conn.row_factory = aiosqlite.Row
1179         async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180             row = await cur.fetchone()
1181             return _row_to_source(row) if row else None
1182

```

```

1183
1184 async def insert_video(conn: aioredis.Connection, video: VideoRecord) -> VideoRecord:
1185     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188     if absent), computes the provider-neutral ``dedup_key`` via
1189     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190     ``VideoRecord`` shape is reconstructed from the inserted row.
1191     """
1192     src = await upsert_source(
1193         conn,
1194         Source(
1195             account_id=video.user_id,
1196             provider="telegram",
1197             provider_source_id=str(video.channel_id),
1198             kind="channel",
1199             watched=True,
1200         ),
1201     )
1202     assert src.id is not None # upsert_source populates the surrogate PK
1203     item = ContentItem(
1204         account_id=video.user_id,
1205         source_id=src.id,
1206         provider="telegram",
1207         provider_message_id=str(video.message_id),
1208         content_type="video",
1209         document_id=str(video.document_id) if video.document_id is not None else None,
1210         duration_s=video.duration_s,
1211         width=video.width,
1212         height=video.height,
1213         size_bytes=video.size_bytes,
1214         mime_type=video.mime_type,
1215         dedup_key=compute_dedup_key(
1216             provider="telegram",
1217             document_id=(str(video.document_id) if video.document_id is not None else
1218             None),
1219             source_id=src.id,
1220             provider_message_id=str(video.message_id),
1221             size_bytes=video.size_bytes,
1222             duration_s=video.duration_s,
1223             height=video.height,
1224             width=video.width,
1225         ),
1226         matched=video.matched,
1227         seen_at=video.seen_at,
1228     )
1229     item = await insert_content_item(conn, item)
1230     return _content_item_to_video(item, src, video.user_id, video.channel_id)
1231
1232 async def mark_forwarded(conn: aioredis.Connection, video_id: int) -> None:
1233     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1234
1235     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1236     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1237     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1238     delivery still targets the single ``delivery_target`` and retried delivery
1239     is harmless (forwards are idempotent). This shim is retained so existing

```

5. assistant (2026-07-06T06:05:09.156Z)

Now let me look at `compute\_dedup\_key` to confirm the document-branch-wins logic, and run a repro.

6. user (2026-07-06T06:05:10.431Z)

```

254: def compute_dedup_key(
255-     *,
256-     provider: str,
257-     document_id: str | None = None,
258-     source_id: int | str | None = None,
259-     provider_message_id: str | None = None,
260-     size_bytes: int | None = None,
261-     duration_s: int | None = None,
262-     height: int | None = None,
263-     width: int | None = None,
264-) -> str:
265-     """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
266-
267-     Implements the ADR 0011 Option 1A decision tree:
268-
269-     - **Telegram** (``provider == "telegram"``):
270-       1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
271-       2. Else if **all four** fingerprint fields (size, duration, height, width)
272-          are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
273-       3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
274-          ``source_id`` is required because Telegram ``message_id`` is only
275-          unique within a channel).
276-     - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
277-       ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
278-       providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
279-
280-     Raises ``ValueError`` if the fallback branch is reached without the fields
281-     it requires (``source_id`` + ``provider_message_id``) ? a degenerate
282-     ``dedup_key`` is worse than a loud failure at ingestion time.
283-     """
284-     p = provider.lower()
285-     if p == "telegram":
286-         if document_id is not None and document_id != "":
287-             return f"tg:doc:{document_id}"
288-         # All four fingerprint fields are required to avoid a degenerate
289-         # ``tg:fp:0:0:0:0`` key when some metadata is missing.
290-         if None not in (size_bytes, duration_s, height, width):
291-             return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292-         # Tier-0 fallback.
293-         if source_id is None or provider_message_id is None:
294-             raise ValueError(

```

7. assistant (2026-07-06T06:05:15.669Z)

The document branch does win in `compute\_dedup\_key` (db.py:286-287). This confirms the mechanism the finding describes. Now let me run the repro to verify the actual behavior, and check v1 on main.

8. user (2026-07-06T06:05:18.571Z)

```

conn.row_factory = aisqlite.Row
async with conn.execute(
    """SELECT * FROM videos
       WHERE user_id = ? AND document_id = ?""",
    (user_id, document_id),
) as cur:
    row = await cur.fetchone()
    return _row_to_video(row) if row else None

async def get_video_by_content_fingerprint(
    conn: aisqlite.Connection,
    user_id: int,
    size_bytes: int,
    duration_s: int,

```

```

    height: int,
    width: int,
) -> VideoRecord | None:
    """Tier-2 content dedup: metadata fingerprint match ? probably same video.

```

Catches re-uploads where Telegram assigns a new ``document.id`` but the file bytes are identical. Uses (size_bytes, duration_s, height, width) as a composite key ? the odds of two different videos sharing all four values are negligible for this use case.

```

"""
conn.row_factory = aiomysql.Row
async with conn.execute(
    """SELECT * FROM videos
       WHERE user_id = ?
          AND size_bytes = ?
          AND duration_s = ?
          AND height = ?
          AND width = ?
       LIMIT 1""",
    (user_id, size_bytes, duration_s, height, width),
) as cur:
    row = await cur.fetchone()
    return _row_to_video(row) if row else None

```

```

async def insert_video(conn: aiomysql.Connection, video: VideoRecord) -> VideoRecord:

```

9. assistant (2026-07-06T06:05:22.525Z)

Confirmed: v1 queried raw `size\_bytes/duration\_s/height/width` columns, independent of `document\_id`. So v1 would catch a re-upload with a new document_id but matching fingerprint. Now let me run the actual repro on the venv.

10. assistant (2026-07-06T06:05:28.715Z)

```

import asyncio
import aiomysql
from tg_video_filter import db
from tg_video_filter.models import VideoRecord

```

```

USER_ID = 42
CHANNEL_ID = 999

```

```

async def main():
    conn = await aiomysql.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_account(conn, USER_ID, "sess")

    # A video seen WITH a document_id AND full fingerprint metadata.
    await db.insert_video(conn, VideoRecord(
        user_id=USER_ID, channel_id=CHANNEL_ID, message_id=5,
        document_id=555, size_bytes=1000, duration_s=60, height=1080, width=1920,
    ))

    # Re-upload arrives: new document_id (111) but identical fingerprint.
    # Ingest path checks Tier-2 by fingerprint.
    found = await db.get_video_by_content_fingerprint(conn, USER_ID, 1000, 60, 1080, 1920)
    print("v2 shim fingerprint hit:", found)

    # What is the stored dedup_key?
    conn.row_factory = aiomysql.Row
    async with conn.execute("SELECT dedup_key FROM content_items") as cur:
        rows = await cur.fetchall()
    print("stored dedup_keys:", [r["dedup_key"] for r in rows])
    await conn.close()

```

```
asyncio.run(main())
```

11. user (2026-07-06T06:05:28.715Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

12. user (2026-07-06T06:07:38.349Z)

Command running in background with ID: b7xpcd4t1. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b7xpcd4t1.output. You will be notified when it completes. To check interim output, use Read on that file path.

13. user (2026-07-06T06:07:41.843Z)

```
1      Traceback (most recent call last):
2      File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 25, in
<module>
3          asyncio.run(main())
4          ~~~~~^~~~~~
5      File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 195, in run
6          return runner.run(main)
7          ~~~~~^~~~~~
8      File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 118, in run
9          return self._loop.run_until_complete(task)
10         ~~~~~^~~~~~
11      File
"C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725,
in run_until_complete
12          return future.result()
13         ~~~~~^~~~~~
14      File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 12, in main
15          await db.upsert_account(conn, USER_ID, "sess")
16         ~~~~~^~~~~~
17      TypeError: upsert_account() takes 2 positional arguments but 3 were given
18
```

14. assistant (2026-07-06T06:07:43.880Z)

Let me check the signature of `upsert\_account`.

15. user (2026-07-06T06:07:45.219Z)

```
730:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
731-    """Insert or update an account row; returns the persisted Account."""
732-    await conn.execute(
733-        """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
734-        VALUES (?, ?, ?, ?, ?)
735-        ON CONFLICT(id) DO UPDATE SET
736-            telegram_id=excluded.telegram_id,
737-            provider=excluded.provider,
738-            display_name=excluded.display_name""",
739-        (
740-            account.id,
741-            account.telegram_id,
742-            account.provider,
743-            account.display_name,
744-            account.created_at.isoformat(),
745-        ),
```



```

198         user.id,
199         document_id,
200         channel_id,
201         message_id,
202         existing.channel_id,
203         existing.message_id,
204     )
205     return
206
207     # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208     if (
209         meta.size_bytes is not None
210         and meta.duration_s is not None
211         and meta.height is not None
212         and meta.width is not None
213     ):
214         existing = await db.get_video_by_content_fingerprint(
215             conn,
216             user.id,
217             meta.size_bytes,
218             meta.duration_s,
219             meta.height,
220             meta.width,
221         )
222         if existing is not None:
223             log.info(
224                 "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225                 "channel=%s msg=%s original_channel=%s original_msg=%s",
226                 user.id,
227                 meta.size_bytes,
228                 meta.duration_s,
229                 meta.height,

```

22. user (2026-07-06T06:08:09.320Z)

```

338:         document_id=555000111,
361:         document_id=123456789,
372:         document_id=123456789,
378:         """Multiple rows with document_id=NULL do NOT collide (partial unique index)."""
386:         document_id=None,
396:         document_id=None,
402:async def test_get_video_by_content_fingerprint_hit(conn: aiosqlite.Connection) -> None:
416:     found = await db.get_video_by_content_fingerprint(
428:async def test_get_video_by_content_fingerprint_miss(conn: aiosqlite.Connection) -> None:
432:     await db.get_video_by_content_fingerprint(
855:     VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=1, document_id=999),
861:     VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=2,
document_id=999),
866:     VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=3, document_id=None),
870:     VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=4, document_id=None),
881:     document_id="12345",
930:     document_id="",
961:     db.compute_dedup_key(provider="TELEGRAM", document_id="5")
962:     == db.compute_dedup_key(provider="telegram", document_id="5")
1031:     document_id="999",
1037:     dedup_key=db.compute_dedup_key(provider="telegram", document_id="999"),
1065:     document_id="777",
1077:     document_id="777",

```

23. assistant (2026-07-06T06:08:16.858Z)

The caller in telethon_client.py (lines 191-221) is exactly as the finding describes: Tier-1 by document_id, then Tier-2 by fingerprint. The `get_video_by_content_fingerprint` test at line 402 seeds a video with NO document_id (the VideoRecord at 404-414 omits `document_id`, defaulting to None), so its dedup_key becomes `tg:fp:...` and the shim finds it. No test exercises the case

where a prior item has a document_id and is then looked up by fingerprint.

Let me confirm the fingerprint test's seed truly lacks a document_id (it does ? lines 404-414 show no document_id field) and verify none of the other document_id tests do a cross-tier fingerprint lookup. Lines 855-870 (test around document_id=999) ? let me check that test to be thorough.

24. user (2026-07-06T06:08:17.918Z)

```
848     async def test_document_id_unique_index_enforces_dedup(conn: aiomysqlite.Connection) ->
None:
849         """The partial unique index must reject a second row with the same document_id."""
850         import sqlite3
851
852         await _seed_user(conn)
853         await db.insert_video(
854             conn,
855             VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=1,
document_id=999),
856         )
857         # Same document_id, different (channel, message) ? must violate the index.
858         with pytest.raises(sqlite3.IntegrityError, match="UNIQUE"):
859             await db.insert_video(
860                 conn,
861                 VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=2,
document_id=999),
862             )
863         # But NULL document_id rows must coexist (partial index).
864         await db.insert_video(
865             conn,
866             VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=3,
document_id=None),
867         )
868         await db.insert_video(
869             conn,
870             VideoRecord(user_id=USER_ID, channel_id=CHANNEL_ID, message_id=4,
document_id=None),
871         )
872
873
874         # --- compute_dedup_key (ADR 0011, Option 1A) -----
875
876
877     def test_compute_dedup_key_telegram_document_id_wins() -> None:
```

25. assistant (2026-07-06T06:08:27.574Z)

That test is about unique-index enforcement, not cross-tier fingerprint dedup. None of the tests seed a video WITH a document_id AND fingerprint metadata and then look it up by fingerprint. The missing-test claim holds.

Now let me weigh the severity honestly. The finding is framed as "missing test," but the substance is a real behavioral regression: v1 caught cross-tier re-uploads (new document_id, same fingerprint); v2 does not, because the fingerprint shim keys only on the `tg:fp` dedup_key while the stored row has `tg:doc:...`. My repro confirmed `None` is returned where v1 returned the row. The trigger is narrow (a re-encoded/re-uploaded file getting a fresh document.id with byte-identical metadata) but realistic for this app's stated purpose ? exactly what v1's Tier-2 existed to catch. The consequence is a re-delivered duplicate, not data loss.

The finding, as titled, is about the missing test. That's accurate ? no test covers it. But the more important truth is the underlying regression is genuine and ships uncaught. Both the test gap and the regression are real and in this diff. Severity: the regression is a real bug with a narrow but realistic trigger and no data loss ? high is defensible, arguably medium given how rare true byte-identical re-uploads with a new document_id are. I'll confirm at high since it's a genuine behavioral regression versus v1 that the test suite fails to catch, matching the

claimed severity.

26. user (2026-07-06T06:08:39.897Z)

Structured output provided successfully